

Programação Funcional

Fundamentos

Marco A L Barbosa

malbarbo.pro.br

Começando

- 1) O que é um literal?
- 2) O que é uma função primitiva?
- 3) Escreva três expressões em Gleam, cada uma correspondente a um caso diferente da definição de expressão. Indique a qual caso cada uma corresponde.
- 4) O que significa avaliar uma expressão?
- 5) Qual é a regra de avaliação para uma chamada de função?
- 6) Qual é a regra de avaliação para uma expressão `case`?
- 7) Escreva uma definição de constante e uma definição de função em Gleam. O que cada uma delas acrescenta ao ambiente?
- 8) O que é uma definição local? Até onde vale o nome que ela define?
- 9) O que é uma função?
- 10) A ordem em que as expressões em uma chamada de função são avaliadas pode alterar o valor da chamada da função? Explique.
- 11) Qual é a diferença entre avaliação em ordem aplicativa e avaliação em ordem normal? Qual das duas o Gleam usa?
- 12) Explique por que `&&` e `||` são formas especiais e não funções. Dê um exemplo de expressão cujo comportamento mudaria se elas fossem funções.
- 13) Quando dois valores são considerados iguais em Gleam? Dê um exemplo de uma comparação com `==` que não compila e explique por quê.
- 14) O que é uma definição com autorreferência? É um processo recursivo?

Praticando

- 15) Determine, sem usar o computador, o valor de cada expressão a seguir. Depois crie um arquivo com as definições abaixo, carregue o arquivo na janela de interações e confira as respostas.

```
pub const a = 7
pub const b = 2
```

- a) `a + b * 3`
 - b) `{ a + b } * 3`
 - c) `a / b`
 - d) `a % b + 1`
 - e) `a - b - 1`
- 16) Reescreva cada expressão a seguir na forma prefixa, isto é, usando apenas chamadas das funções `int.add`, `int.subtract` e `int.multiply`, sem alterar o valor produzido.

- a) `2 + 3 * 5`
- b) `{ 2 + 3 } * 5`
- c) `10 - 7 + 6`
- d) Por que `10 / 2` não pode ser reescrita da mesma forma, com `int.divide`?

17) Determine, sem usar o computador, o valor de cada expressão a seguir. Depois crie um arquivo com as definições abaixo, carregue o arquivo na janela de interações e confira as respostas.

```
pub const x = 3
pub const y = 4
```

- a) `x > y || x % 2 == 1`
- b) `!{ x > y }`
- c) `x > 0 && y / x > 1`
- d) `x + y * 2 > 10 && x != y`
- e) `x < y == True`

18) Faça uma função chamada `area_retangulo` que recebe dois argumentos, a `largura` e a `altura` de um retângulo, e calcula a sua área. Use o método de substituição para verificar se a função funciona corretamente de acordo com os exemplos a seguir. Confira as respostas na janela de interações.

```
> area_retangulo(3.0, 5.0)
15.0
> area_retangulo(2.0, 2.5)
5.0
```

19) Faça uma função chamada `produto_anterior_posterior` que recebe um número inteiro `n` e calcula o produto de `n`, `n + 1` e `n - 1`. Use o método de substituição para verificar se a função funciona corretamente de acordo com os exemplos a seguir. Confira as respostas na janela de interações.

```
> produto_anterior_posterior(3)
24
> produto_anterior_posterior(1)
0
> produto_anterior_posterior(-2)
-6
```

20) Reescreva a função a seguir sem usar `let`, criando uma função auxiliar. Explique por que as duas versões produzem o mesmo resultado.

```
fn imc(massa: Float, altura: Float) -> Float {
  let altura2 = altura *. altura
  massa /. altura2
}
```

21) Considere as definições a seguir.

```
fn dobro(x) {
  x + x
}

fn g(a, b) {
  dobro(a) + b
}
```

- a) Escreva os passos da avaliação de `g(2 + 1, 4)` usando a ordem aplicativa.
- b) Escreva os passos da avaliação de `g(2 + 1, 4)` usando a ordem normal.
- c) Em cada uma das ordens, quantas vezes a expressão `2 + 1` é reduzida? Explique a diferença.

22) Faça uma função chamada `eh_par` que recebe um número natural `n` e indica se `n` é par. Um número é par se o resto da divisão dele por 2 é igual a zero. Não use `case` e nem a função pré-definida

`int.is_even`. Use o método de substituição para verificar se a função funciona corretamente de acordo com os exemplos a seguir. Confira as respostas na janela de interações.

```
> eh_par(3)
False
> eh_par(6)
True
```

- 23) Faça uma função chamada `tem_tres_digitos` que recebe um número natural `n` e verifica se `n` tem exatamente 3 dígitos. Não use `case`. Use o método de substituição para verificar se a função funciona corretamente de acordo com os exemplos a seguir. Confira as respostas na janela de interações.

```
> tem_tres_digitos(99)
False
> tem_tres_digitos(100)
True
> tem_tres_digitos(999)
True
> tem_tres_digitos(1000)
False
```

- 24) Faça uma função chamada `entre` que recebe três inteiros, `a`, `x` e `b`, e determina se `x` está entre `a` e `b`, incluindo os extremos. Não use `case`. Use o método de substituição para verificar se a função funciona corretamente de acordo com os exemplos a seguir. Confira as respostas na janela de interações.

```
> entre(2, 5, 8)
True
> entre(2, 2, 8)
True
> entre(2, 9, 8)
False
```

- 25) Faça uma função `maximo` que encontre o máximo entre dois inteiros. Não use a função `int.max`. Use o método de substituição para verificar se a função funciona corretamente de acordo com os exemplos a seguir. Confira as respostas na janela de interações.

```
> maximo(3, 5)
5
> maximo(8, 4)
8
> maximo(6, 6)
6
```

- 26) Faça uma função chamada `ordem` que recebe três inteiros distintos, `a`, `b` e `c` e determina se a sequência `a`, `b`, `c` está em ordem crescente, decrescente ou não está em ordem. Use o método de substituição para verificar se a função funciona corretamente de acordo com os exemplos a seguir. Confira as respostas na janela de interações.

```
> ordem(3, 8, 12)
" crescente "
> ordem(3, 1, 4)
" sem ordem "
> ordem(3, 1, 0)
" decrescente "
```

Desafios

- 27) Faça uma função chamada `so_primeira_maiuscula` que recebe uma palavra não vazia (string) como parâmetro e crie uma nova string convertendo a primeira letra da palavra para maiúscula e o restante da palavra para minúscula. Use o método de substituição para verificar se a função funciona corretamente

de acordo com os exemplos a seguir. Confira as respostas na janela de interações. Veja as funções `string.slice`, `string.length`, `string.uppercase` e `string.lowercase`.

```
> so_primeira_maiuscula("paula")
"Paula"
> so_primeira_maiuscula("ALFREDO")
"Alfredo"
```

- 28) [sicp 1.4] O modelo de avaliação visto em sala permite que os operadores em chamadas de funções sejam expressões compostas. Use esta observação para descrever o comportamento do seguinte procedimento:

```
fn a_plus_abs_b(a, b) {
  case b > 0 {
    True -> int.add
    False -> int.subtract
  }(a, b)
}
```

- 29) [sicp 1.5] Ben Bitdiddle inventou um método para determinar se um interpretador está usando avaliação com ordem aplicativa ou avaliação com ordem normal. Ele definiu os seguintes procedimentos:

```
fn p() {
  p()
}

fn teste(x, y) {
  case x == 0 {
    True -> 0
    False -> y
  }
}
```

Então avaliou a seguinte expressão

```
teste(0, p())
```

Qual é o comportamento que Ben observará com um interpretador que usa avaliação com ordem aplicativa? Qual é o comportamento que ele observará com um interpretador que usa avaliação com ordem normal? Explique a sua resposta.